PORTAFOLIO TÉCNICO · INFRAESTRUCTURA Y ENTREGA

TRES PRODUCTOS EN GOOGLE CLOUD, DE LA RAMA AL TRÁFICO

Diseño y operación de la infraestructura de tres APIs en producción sobre Cloud Run: contenedores que se verifican antes de publicarse, despliegue automático por repositorio, migraciones deliberadamente fuera del pipeline, y un entorno de preproducción levantado desde cero en un proyecto aparte.

Desarrollo backend e infraestructura RUSHINGWIND 2026

3	4	3	2	1
servicios en Cloud Run, en producción	triggers de Cloud Build (despliegue y verificación de PR)	instancias de Cloud SQL · PostgreSQL 17	proyectos de GCP: producción y staging	Cloud Run Job y 1 Cloud Scheduler

01 LOS PRODUCTOS

Tres bases de código independientes, un solo proyecto de GCP y una sola forma de desplegar. Comparten registro de imágenes, cuenta de servicio y región; no comparten datos: cada una con su instancia, sus roles y sus secretos.

API de contenido

MONOREPO · NestJS + Prisma

El producto grande: API, aplicación móvil, app de televisión y web, en un monorepo con paquetes compartidos.

- Cloud Run + Cloud SQL dedicado
- **Cloud Run Job** para migraciones
- Cloud Scheduler para envíos diarios
- Firebase: identidad, almacenamiento, notificaciones y hosting

Monedero de eventos

REPO PROPIO · NestJS + Prisma

Cobros con pulsera NFC en eventos. Dinero real, así que el pipeline es el más estricto de los tres.

- Cloud Run + Cloud SQL propio
- Imagen multietapa con etapa de verificación
- Un secreto por rol de base de datos
- Migraciones a mano, por decisión

Plataforma de formación

`MONOREPO · API + web`

API en Cloud Run y una web estática que se publica en el mismo pipeline, después de la API.

- Cloud Run + Cloud SQL propio
- Arranque real del contenedor como paso de CI
- Firebase Hosting con *targets*
- Orden forzado: la web nunca llama a una API vieja

02

EL PIPELINE, PASO A PASO

Un archivo por repositorio y un trigger que lo dispara en cada push a la rama principal. Los pasos están ordenados de modo que **nada se publica sin haber sido verificado**, y el fallo aparece en su propio paso en vez de disfrazado de error de compilación.

1 **Verificar, dentro de la imagen**

Una etapa del Dockerfile que solo existe para esto: comprobación de tipos, la batería de pruebas unitarias y un chequeo de que la referencia OpenAPI publicada coincide con el código. Se construye con `--target`, aparte de la imagen final. Si algo falla, el build muere antes de publicar nada.

2 **Construir**

Imagen multietapa: instalar, compilar, verificar, *podar* las dependencias de desarrollo y copiar a una imagen final mínima, sin privilegios y con usuario propio. De cientos de megas a unas decenas — y lo que no viaja no se puede explotar.

3 **Arrancar el contenedor de verdad**

En uno de los proyectos hay un paso que levanta la imagen recién construida y le pide la ruta de salud. Un contenedor que compila pero no arranca es un fallo que de otro modo se descubre en producción, en el despliegue.

4 **Publicar en Artifact Registry**

Dos etiquetas por imagen: el hash corto del commit y `latest`. El despliegue referencia **el hash**, nunca `latest`: así se sabe exactamente qué revisión está sirviendo y se puede volver atrás señalando una imagen concreta.

5 **Desplegar en Cloud Run**

Conexión al Cloud SQL por socket, variables de entorno no sensibles en el propio comando y las sensibles montadas desde Secret Manager. La revisión nueva solo recibe tráfico si arranca y responde; si no, el build falla y el tráfico se queda donde estaba.

> **Y UN SEGUNDO TRIGGER POR REPOSITORIO**
>
> Cada repo tiene también un trigger de **pull request** con un archivo distinto: construye y verifica, pero no publica ni despliega. Es el mismo trabajo sin la parte irreversible, y es lo que hace que la rama principal llegue al despliegue ya revisada.

03 LAS MIGRACIONES NO VAN EN EL PIPELINE

Es la decisión de infraestructura con más consecuencias de todo el conjunto, y conviene explicar por qué.

Lo natural es añadir un paso de migración al despliegue. No se hizo, y el motivo es Cloud Run: **el paso de arranque corre en cada instancia**. Con escalado automático, varias instancias levantando a la vez aplicarían las mismas migraciones en paralelo sobre la misma base. En un sistema que mueve dinero, eso no es un riesgo aceptable.

La migración vive fuera, y de dos maneras según el producto: un **Cloud Run Job** dedicado que se ejecuta una vez por versión, y en el otro un guion que abre un proxy autenticado contra Cloud SQL y aplica lo pendiente con un usuario distinto al de la aplicación.

Sacarlas del pipeline crea una asimetría —el código se despliega solo, el esquema no— y esa asimetría obliga a una regla de orden. Romperla deja una ventana con la API pidiendo algo que todavía no existe:

TIPO DE CAMBIO	ORDEN	POR QUÉ
Aditivo columna o tabla nueva	`migrar → desplegar`	El código nuevo pide la columna en cada consulta; el viejo la ignora sin enterarse.
Destructivo borrar o renombrar	`desplegar → migrar`	Al revés: el código viejo seguiría pidiendo lo que la migración acaba de quitar.

TIPO DE CAMBIO	ORDEN	POR QUÉ
Restricción nueva índice único, validación	`desplegar → migrar`	No añade nada que pedir: añade algo que la base puede *negar*. Al revés, un duplicado devuelve «error interno» a quien lo teclea en vez del conflicto que el código nuevo sabe traducir.
Cambio de tipo o de valor con datos dentro	`los dos seguidos`	Hay ventana en cualquiera de los dos órdenes. Se minimiza, se avisa, y no se hace con un evento en marcha.

Renombrar una columna en producción, por tanto, son **dos despliegues**: añadir la nueva escribiendo en ambas, migrar los datos, y quitar la vieja después. Las migraciones no se revierten, así que esa disciplina es lo único que hace seguro revertir el código.

04 SECRETOS, IDENTIDAD Y PERMISOS

Ninguna credencial vive en el repositorio ni en la imagen. Cada valor sensible es un secreto de **Secret Manager** montado por Cloud Run en el arranque, y hay **un secreto por rol de base de datos**: el que usa la aplicación no es el que migra, y el que migra no es el propietario del esquema.

Los permisos de la base se aplican aparte del esquema y se vuelven a aplicar después de cada migración. Al rol de la aplicación se le **revocan** explícitamente el borrado y la modificación sobre la tabla del libro contable: es de solo inserción por permisos, no por disciplina de quien escribe el código.

La firma de las sesiones se comparte entre dos de las APIs, porque una emite los tokens y la otra solo los verifica. Está documentado como deuda consciente —lo correcto sería un algoritmo asimétrico, con la clave pública del lado que verifica— y el módulo que lo hace está aislado para que ese cambio sea local.

> **LO QUE NO ES EVIDENTE**
>
> `roles/editor` **no** da acceso al contenido de los secretos. Cada secreto necesita su enlace explícito de *accessor* para la cuenta de servicio. Sin él, el servicio despliega y muere al arrancar, con un error que no menciona permisos por ningún lado.

05 OBSERVABILIDAD

Un log que no se puede filtrar es un log que no se lee.

Las tres APIs escribían texto plano, y en Cloud Logging eso significa que **todo llega con severidad informativa, los errores incluidos**: no hay campo por el que filtrar ni nada sobre lo que alertar. Se sustituyó por registro estructurado en JSON con los nombres que la plataforma trata como suyos —severidad, mensaje, marca de tiempo— de modo que la consola clasifica, filtra y alerta sin trabajo adicional.

Lo que más valor dio no fue el formato sino la **correlación**: cada petición lleva un identificador que se reutiliza del propio encabezado de traza de la plataforma cuando existe, viaja de vuelta en la respuesta y se adjunta a *todas* las líneas de esa petición, incluidas las que escribe la lógica de negocio. El resultado práctico: el identificador que reporta un cliente y la traza de la consola son el mismo hilo, y un rechazo de autenticación y el código de estado que lo acompaña dejan de ser dos líneas sueltas.

Dos detalles que se decidieron a conciencia: la sonda de salud **no** se registra —golpea cada pocos segundos y no dice nada— pero una sonda que falla sí; y los encabezados de la petición no se escriben nunca, porque el serializador por omisión de la librería incluye el de autorización, y un registro con eso dentro es una sesión regalada a quien lea la consola.

06 UN ENTORNO DE STAGING DESDE CERO

Proyecto de GCP aparte, no un sufijo en los nombres del de producción.

La razón de que sea un proyecto separado y no otro servicio dentro del mismo es el radio de daño: con proyectos distintos, una prueba no puede tocar la base, el bucket ni los secretos de producción *por accidente*, porque no tiene permiso para verlos.

Se levantó completo y documentado por fases, de forma que otra persona pueda repetirlo: proyecto y facturación, cuenta de servicio con lo justo, base de datos y copia de los datos, secretos propios generados en el momento, bucket de medios, el servicio y su job de migración, el planificador, y por último las aplicaciones apuntadas con un perfil de compilación distinto.

El documento incluye también lo que **no** cubre —las tiendas de aplicaciones, las páginas legales, los servicios de los otros equipos— y el costo mensual estimado, con la alternativa más barata de tamaño de base de datos. Un entorno de pruebas cuyo costo nadie conoce es un entorno que alguien apaga.

07

LO QUE COSTÓ UN FALLO REAL

Cada línea corresponde a algo que se rompió una vez y dejó de romperse porque quedó escrito. Es la parte del trabajo que no se ve en un diagrama.

SÍNTOMA	CAUSA	LO QUE QUEDÓ
El trigger no se crea y la respuesta no dice qué falta	Los triggers de segunda generación exigen indicar la cuenta de servicio de forma explícita	Documentado junto al comando de creación
El proxy contra la base muere al arrancar y el cliente falla contra un puerto vacío	El proxy usa credenciales de aplicación, que son otro archivo y otro comando distintos a los de la sesión normal	Anotado como primer paso del guion de migración
La revisión no levanta y el tráfico se queda en la anterior	El secreto se creó con la salida de un gestor de paquetes, que escribe su propia cabecera por la salida estándar: 192 bytes en vez de 45	Comprobar la longitud antes de subirlo — sin mostrar el valor
El build falla con «la ruta no existe», y en local funciona siempre	Un guion que corre dentro de la imagen ve un *subconjunto* del repositorio: solo lo que esa etapa copia	Al tocar un guion del pipeline, revisar qué copia su etapa
El build dice «código de salida 1» y nada más	Salir del proceso de inmediato trunca la salida cuando no hay terminal; y un arranque con el registro apagado muere sin imprimir el motivo	Código de salida por asignación, y nunca apagar el registro en un guion
La API entera responde error interno al leer saldos	Recrear una vista le quita sus permisos, y los permisos por omisión no cubren objetos recreados	Reaplicar los permisos después de tocar cualquier vista
El servicio arranca y muere al primer uso de la base	El cliente de la base es dependencia de *producción*, no de desarrollo: la poda se lo llevaba por delante	Regenerar el cliente después de podar, en esa etapa

08

VERIFICACIÓN ANTES DE TOCAR PRODUCCIÓN

Ningún cambio de esquema sale del repositorio sin que se haya creado una base **desde cero** aplicando todas las migraciones en orden, otra desde el archivo original de referencia, y comparado restricciones, índices y columnas entre las dos. Esa comparación es barata y ha encontrado cosas que ninguna prueba veía: restricciones que comparaban mal y habrían

hecho imposible una operación normal del negocio, y una regla de integridad copiada con la columna equivocada.

Encima de eso, cada despliegue va precedido de comprobación de tipos, pruebas unitarias, pruebas de integración contra una base real —incluidas las de concurrencia, que disparan peticiones en paralelo y exigen que solo una gane— y un recorrido de humo que ejercita la API de punta a punta y hoy suma 350 comprobaciones.

09 TECNOLOGÍAS

GOOGLE CLOUD

- Cloud Run (servicios y jobs)
- Cloud Build y triggers
- Artifact Registry
- Cloud SQL · PostgreSQL 17
- Secret Manager
- Cloud Scheduler
- Cloud Logging y Cloud Trace
- IAM y cuentas de servicio

FIREBASE

- Authentication
- Cloud Storage y sus reglas
- Cloud Messaging
- Hosting con *targets*

PLATAFORMA

- Docker multietapa
- Node.js 22 · TypeScript
- NestJS · Prisma
- pnpm y monorepos
- Jest: unitarias, integración y concurrencia
- OpenAPI generado y verificado

PRÁCTICAS

- Migraciones SQL escritas a mano
- Orden de despliegue por tipo de cambio
- Registro estructurado y correlación de trazas
- Verificación de PR sin publicar
- Documentación de decisiones, no de comandos

Documento de portafolio. Describe arquitectura y decisiones de trabajo realizado en RUSHINGWIND durante 2026; se omiten a propósito identificadores de infraestructura — proyectos, cuentas de servicio, instancias, nombres de secretos y direcciones de servicio—, que no aportan al relato y no deben publicarse.